模块八：Docker 容器化部署

学习目标

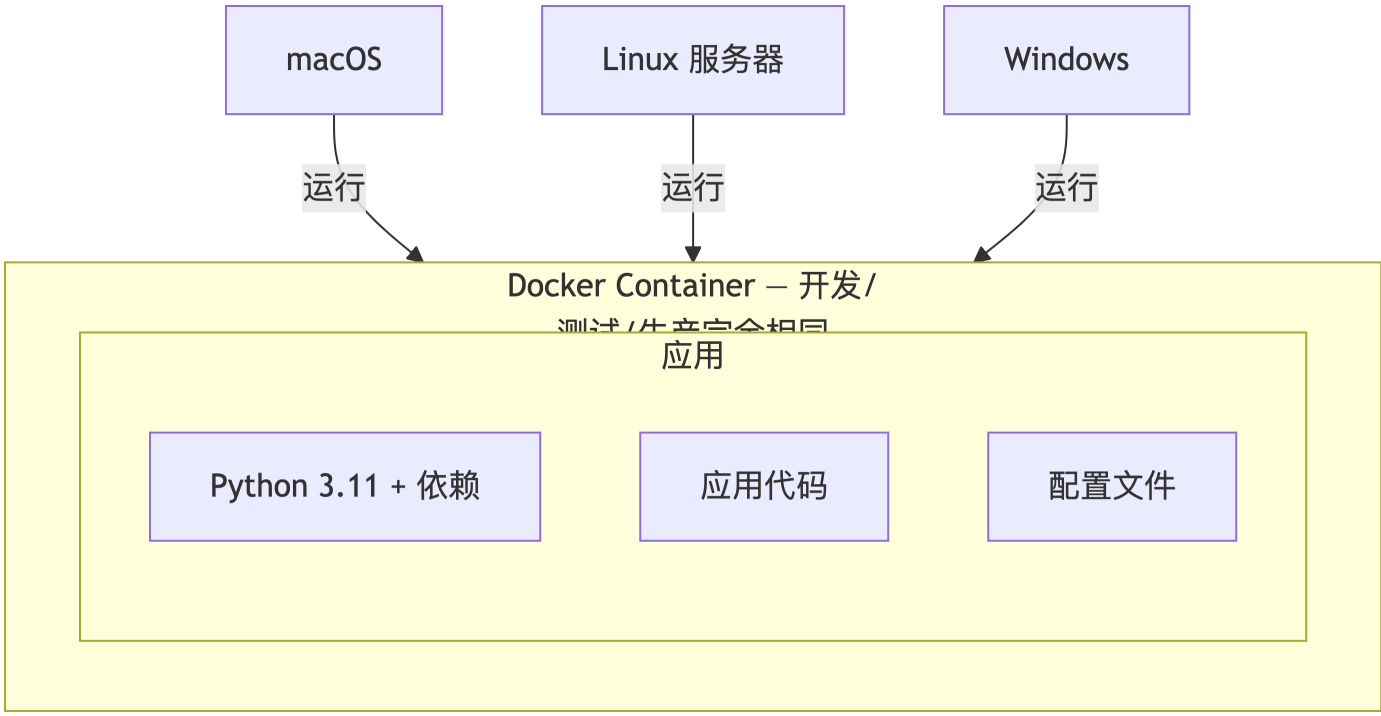
- 理解容器化的核心价值
- 掌握 Dockerfile 的编写（多阶段构建）
- 学会用 Docker Compose 编排多服务
- 了解基本的部署流程

8.1 为什么需要容器化？

“It Works on My Machine” 问题

不用 Docker 部署时，常见的问题:

Docker 的解决方案



核心概念对比

概念	类比	说明
Image（镜像）	安装光盘	包含 OS + 依赖 + 代码的只读模板
Container（容器）	运行中的虚拟机	Image 的运行实例
Dockerfile	安装脚本	构建 Image 的步骤说明
Docker Compose	集群部署脚本	同时启动多个容器
Volume（卷）	外部硬盘	容器重启后数据不丢失

8.2 Dockerfile 详解

我们的 Dockerfile

```
# ==========================================
# Stage 1: Builder - 安装依赖
# ==========================================
FROM python:3.11-slim AS builder

WORKDIR /build
```

```dockerfile
COPY requirements.txt .

# 创建虚拟环境并安装依赖
RUN python -m venv /opt/venv && \
    /opt/venv/bin/pip install --no-cache-dir -r requirements.txt

# ==========================================
# Stage 2: Runtime - 运行应用
# ==========================================
FROM python:3.11-slim AS runtime

WORKDIR /app

# 从 builder 阶段复制已安装的依赖
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# 复制应用代码
COPY src/ ./src/
COPY data/ ./data/
COPY api_server.py .
COPY app.py .
COPY streamlit_app.py .

# 非 root 用户（安全实践）
RUN useradd --create-home appuser
USER appuser

EXPOSE 8000 8501

# 健康检查
HEALTHCHECK --interval=30s --timeout=10s --start-period=15s --retries=3 \
    CMD python -c "import httpx; httpx.get('http://localhost:8000/api/v1/
health').raise_for_status()"

CMD ["python", "api_server.py", "--host", "0.0.0.0"]
```

关键概念解析

多阶段构建

好处: 最终镜像不包含构建工具，体积更小。

层缓存策略

```
#  好：requirements.txt 很少改变，充分利用缓存
COPY requirements.txt .         # 第一层：依赖声明
RUN pip install -r ...          # 第二层：安装依赖（缓存！）
COPY src/ ./src/                # 第三层：代码（经常变化）

#  差：每次代码改变都要重新安装依赖
COPY . .                        # 所有文件一起复制
RUN pip install -r ...          # 每次都重新安装
```

安全实践

```
# 1. 非 root 用户运行
RUN useradd --create-home appuser
USER appuser
# → 即使容器被攻破，攻击者也没有 root 权限

# 2. .dockerignore 排除敏感文件
# .env 文件不会被复制到镜像中
# → 密钥通过环境变量在运行时注入
```

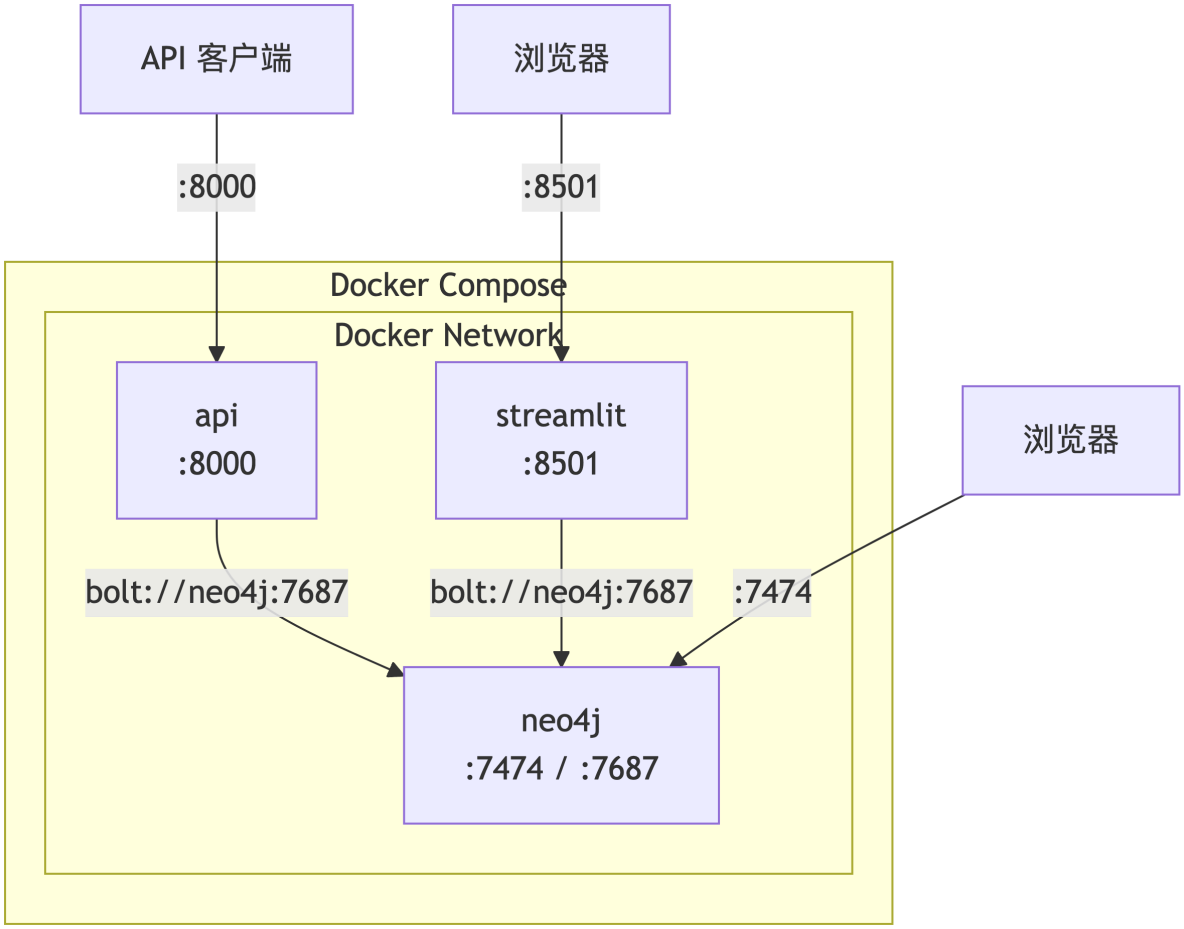
HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=15s --retries=3 \
    CMD python -c "import httpx; httpx.get('http://localhost:8000/api/v1/
health').raise_for_status()"
```

Docker 每 30 秒调用一次健康检查端点: - `healthy` — 服务正常 - `unhealthy` — 服务异常（可配置自动重启）

8.3 Docker Compose 编排

服务架构



docker-compose.yml 解析

```yaml
services:
  # 1. Neo4j 数据库
  neo4j:
    image: neo4j:5-community          # 使用官方社区版镜像
    ports:
      - "7474:7474"                    # Browser UI
      - "7687:7687"                    # Bolt 协议
    environment:
```

```yaml
      NEO4J_AUTH: neo4j/${NEO4J_PASSWORD:-password123}
    volumes:
      - neo4j_data:/data                # 数据持久化！
    healthcheck:                         # 健康检查
      test: ["CMD-SHELL", "wget ... http://localhost:7474"]
      interval: 10s

  # 2. FastAPI 服务
  api:
    build: .                            # 从 Dockerfile 构建
    ports:
      - "8000:8000"
    environment:
      NEO4J_URI: bolt://neo4j:7687      # 用服务名作为主机名
      OPENAI_API_KEY: ${OPENAI_API_KEY}
    depends_on:
      neo4j:
        condition: service_healthy      # 等 Neo4j 健康后再启动

  # 3. Streamlit UI
  streamlit:
    build: .
    ports:
      - "8501:8501"
    depends_on:
      neo4j:
        condition: service_healthy
    command: ["streamlit", "run", "streamlit_app.py", "--server.address",
"0.0.0.0"]
```

关键概念

服务依赖 (depends_on)

```yaml
api:
  depends_on:
    neo4j:
      condition: service_healthy    # 不仅是"启动了"，还要"健康了"
```

启动顺序： Neo4j 启动 → 健康检查通过 → API 启动 → Streamlit 启动

数据持久化 (volumes)

```yaml
volumes:
  neo4j_data:    # 命名卷：Neo4j 数据在容器重建后保留
  neo4j_logs:    # 日志也持久化
```

不用 **volume** 的后果: `docker compose down` 后所有数据丢失！

Docker 网络

容器之间通过服务名通信: - API 连接 Neo4j: `bolt://neo4j:7687`（不是 `localhost`！）- 这是因为 Docker Compose 自动创建了内部网络

环境变量传递

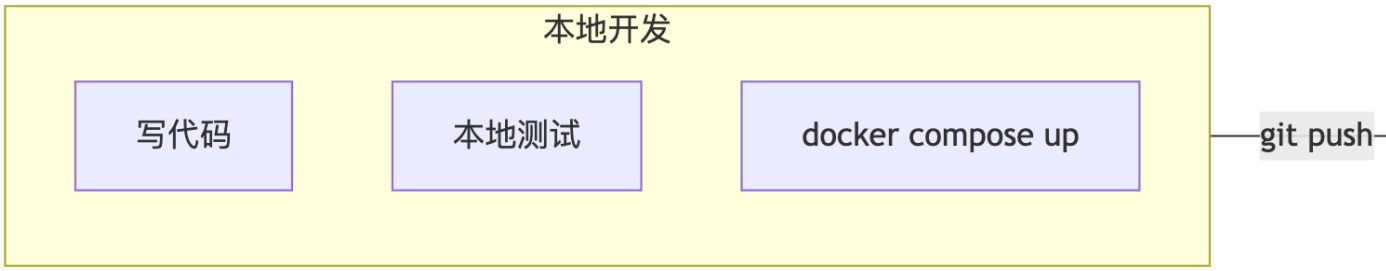
```yaml
environment:
  OPENAI_API_KEY: ${OPENAI_API_KEY}    # 从宿主机 .env 文件读取
  NEO4J_PASSWORD: ${NEO4J_PASSWORD:-password123}  # 带默认值
```

8.4 部署流程

本地部署

```bash
# 1. 准备环境变量
cp .env.example .env
# 编辑 .env，填入 API Key

# 2. 启动所有服务
docker compose up -d

# 3. 查看状态
docker compose ps

# 4. 加载种子数据（首次）
# 打开 http://localhost:7474，执行 seed_graph.cypher

# 5. 测试
curl http://localhost:8000/api/v1/health
open http://localhost:8501

# 6. 查看日志
docker compose logs -f api

# 7. 停止
docker compose down
```

云服务器部署概念



云部署的基本步骤: 1. 在云服务器安装 Docker + Docker Compose 2. 将代码上传到服务器（git clone）3. 配置 `.env`（API Key 等）4. `docker compose up -d` 5. 配置域名和 HTTPS（可选）

8.5 动手实验

实验 1: 构建和运行单个容器

```
# 构建镜像
docker build -t smart-home-agent .

# 查看镜像大小
docker images smart-home-agent

# 运行容器（需要先有 Neo4j）
docker run -p 8000:8000 --env-file .env smart-home-agent
```

实验 2: Docker Compose 完整部署

```
# 启动所有服务
docker compose up

# 观察启动顺序: Neo4j → API → Streamlit
# 观察日志中的 health check

# 在新终端测试
curl http://localhost:8000/api/v1/health
curl http://localhost:8000/api/v1/graph/stats
```

实验 3: 理解 Volume 持久化

```
# 加载数据
# ... (通过 Neo4j Browser 执行 seed_graph.cypher)

# 重建容器（不删除 volume）
docker compose down
```

```
docker compose up -d

# 数据还在吗？
curl http://localhost:8000/api/v1/graph/stats
# → 应该仍然有数据！

# 删除 volume（数据会丢失！）
docker compose down -v
```

关键文件

文件	内容
`Dockerfile`	多阶段构建配置
`docker-compose.yml`	服务编排
`.dockerignore`	构建排除列表
`.env.example`	环境变量模板

下一节：09_高级 Agent 模式.qmd